Laboratory Exercise – Reinforcement Learning and Deep RL

Using an LLM as an Engineering Assistant

Artificial Intelligence

Contents

1 Learning Objectives **2**

2 Background **2**

2.1 Tabular Q-learning 2

2.2 Deep Q-Networks 2

3 Software Requirements **3**

4 Scenario: Robots in a Warehouse **3**

4.1 Rewards and stopping rules 4

5 Part I: Design and Test a Q-learning Agent **5**

6 Part II: Replace the Table with a Neural Network **8**

7 Part III: Two Robots and Collision Avoidance **9**

8 Evaluation and Reflection **11**

8.1 Optional: Connect the Exercise to RLHF 12

9 Submission **12**

10 Reading **13**

1 Learning Objectives

By the end of this laboratory, you should be able to:

1. formulate a robot-navigation problem as an MDP;
2. identify states, actions, transitions, rewards, and terminal outcomes;
3. use an LLM to help implement tabular Q-learning;
4. test the environment and the learning update separately;
5. explain epsilon-greedy exploration;
6. replace a Q-table with a Deep Q-Network (DQN);
7. extend the controller to two robots and test collision handling;
8. inspect, test, and explain an LLM-generated learning system.

2 Background

In the lectures, we studied how an agent can learn from rewards. At each step, the agent observes a state, chooses an action, and receives a reward and a next state.

An MDP specifies

$$(S, A, p, R, \gamma),$$

where S is the state space, A is the action space, p describes the transitions, R specifies rewards, and γ is the discount factor. In this laboratory, movement is deterministic.

2.1 Tabular Q-learning

A Q-table stores an estimated action value $Q(s, a)$ for each state–action pair. After observing (s, a, r, s'), form the target

$$y = \begin{cases} r, & \text{if the transition truly terminates the task,} \\ r + \gamma \max_{a'} Q(s', a'), & \text{otherwise.} \end{cases}$$

Then update only the entry for the action taken:

$$Q(s, a) \leftarrow Q(s, a) + \alpha(y - Q(s, a)).$$

During training, use epsilon-greedy action selection: with probability ε, choose uniformly from all actions; otherwise choose an action with the largest current Q-value. Break ties randomly during training.

2.2 Deep Q-Networks

DQN replaces the table with a neural network Q_θ. The network receives a representation of the state and outputs one value per action. These outputs are Q-values, not probabilities.

For a batch of transitions, use

$$y_i = r_i + \gamma(1 - d_i) \max_{a'} Q_{\theta^-}(s_i', a'), \qquad L(\theta) = \frac{1}{B} \sum_{i=1}^{B} \big(Q_\theta(s_i, a_i) - y_i\big)^2.$$

Here $d_i = 1$ only for true termination, B is the batch size, and Q_{θ^-} is the target network.

Recall the two supporting ideas from the lecture:

- **Experience replay:** store transitions and sample random batches from that stored experience.
- **Target network:** compute the targets with a separate copy of the network, whose weights are updated periodically.

Important

You will design the learning system and use an LLM to help implement it. You must check that the generated program follows your specification.
A plausible trajectory is not enough. Test the movement rules, rewards, Q-learning targets, and collision rules before interpreting training results.

3 Software Requirements

You will need:

- Python 3.10 or later;
- a Python environment such as Jupyter, VS Code, or a terminal;
- NumPy and Matplotlib;
- PyTorch for the DQN part;
- access to an LLM, such as ChatGPT, Gemini, Claude, or Copilot.

For a local installation, you may use:

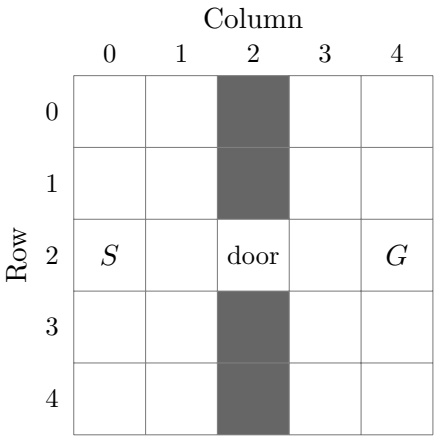
```
python -m pip install numpy matplotlib torch
```

A CPU is sufficient for the small networks used here. Implement the environment and learning loops with the LLM's assistance. Use NumPy for the Q-table and PyTorch for DQN; avoid a ready-made RL training library so that you can inspect the updates.

4 Scenario: Robots in a Warehouse

A warehouse has two rooms connected by one narrow doorway. A robot must move from the left room to a delivery point in the right room.

The warehouse is a 5×5 grid. Positions are written as (r, c), where rows and columns run from 0 to 4. Rows increase downwards.



The shaded cells are obstacles:

$$\{(0,2),(1,2),(3,2),(4,2)\}.$$

All other cells are free. The initial position is $S = (2,0)$ and the goal is $G = (2,4)$.

Each action takes one time step:

$$\text{Up}, \quad \text{Down}, \quad \text{Left}, \quad \text{Right}, \quad \text{Wait}.$$

Use action codes $0, 1, 2, 3, 4$, in that order. A move into an obstacle or beyond the grid leaves the robot in place. Waiting is a valid action.

4.1 Rewards and stopping rules

For the single robot, use the following mutually exclusive reward rules.

Outcome of the action	Reward
The robot reaches its goal	$+10$
The robot attempts an obstacle or boundary move	-1
Any other valid action, including waiting	-0.1

Reaching the goal terminates the task. Use $\gamma = 0.95$. Stop an unfinished rollout after 40 steps so that a run cannot continue forever.

Important

Distinguish true termination from the 40-step collection cutoff. Reaching the goal is termination. The cutoff is truncation.
Stop the rollout on either event, but suppress the continuation term in a TD target only for true termination. On truncation, use the last next state before resetting. This cutoff is not a deadline in the underlying task.

The map, goal, and reward rules remain fixed during each experiment. The learner receives no path demonstrations and is not given a path by a search algorithm.

5 Part I: Design and Test a Q-learning Agent

Task 1: Understand the MDP

Before writing code or asking an LLM for an implementation, formulate the single-robot problem as an MDP.

Component	Your specification
State	
Actions	
Transition rule	
Reward	
Initial state	
Terminal outcome	

Then answer:

(a) What information is necessary to specify the state?

(b) Why is the current position sufficient when the map and goal are fixed?

(c) Is the transition rule deterministic?

(d) What is the shortest valid route from S to G?

(e) Is finding that route by search the same process as learning action values from experience?

Think About It

First specify the task. Then choose an implementation. The LLM should receive your design, rather than decide the meaning of states, rewards, and success on your behalf.

Task 2: Plan the Learning Agent

Write a short design before prompting the LLM. Specify:

1. how the grid and robot position will be represented;
2. how `reset` and `step` will work;
3. how the Q-table will be indexed;
4. how epsilon-greedy action selection will be implemented;
5. how the terminal flag will enter the Q-learning target;

6. when an episode will stop;
7. how training and evaluation will differ.

A convenient table has shape 25×5: cell (r, c) has index $5r + c$. Entries for obstacle cells can remain unused.
Decide what the program should record. At a minimum:

- training episode return and whether the goal was reached;
- the value of ε during training;
- the final greedy trajectory and its length;
- invalid moves, success, and timeouts.

Think About It

During training the robot needs to try actions. During evaluation, you want to inspect the learned policy. Specify how to disable exploration and parameter updates during evaluation.

Task 3: Ask an LLM to Implement Q-learning

Use your design to ask an LLM for a simple implementation. Include the exact map, coordinates, actions, rewards, and stopping rules.
You may begin with the following prompt and adapt it.

Example prompt

Implement tabular Q-learning in Python using NumPy for the warehouse environment specified in this laboratory.
Use a 5×5 grid, the obstacle set $\{(0, 2), (1, 2), (3, 2), (4, 2)\}$, start $(2, 0)$, and goal $(2, 4)$. Actions are up, down, left, right, and wait. An obstacle or boundary move stays in place. Use mutually exclusive rewards: $+10$ on reaching the goal, -1 for an invalid move, and -0.1 for another valid action.
Provide separate functions for the environment, epsilon-greedy action selection, one Q-learning update, training, and greedy evaluation. Use $\alpha = 0.2$, $\gamma = 0.95$, an initially zero Q-table, and seed 7. Train for 1,500 episodes. Decrease epsilon linearly from 1 to 0.05 over the first 1,200 episodes.
Cut off unfinished episodes after 40 steps. Return separate termination and truncation flags. Bootstrap at truncation, but not at goal termination. Use the next state from before reset in every update.
Do not use path search or a ready-made RL library. Include a separate test mode in which I can set the robot's position, choose one action, and inspect the outcome without training. Report training curves and the learned greedy trajectory. Keep the implementation simple and explain its main functions.

Read the generated code before running a long training session. If the LLM changes a rule or uses a term you do not understand, ask for an explanation and revise the program. Save the important prompts and any corrections you request.

Task 4: Test the Environment and the Q-learning Update

Test the environment without a learner first. Set the initial position for each test explicitly.

Position	Action	Expected result
$(0, 0)$	up	Stay at $(0, 0)$; reward -1; no termination.
$(1, 1)$	right	Stay at $(1, 1)$; reward -1; no termination.
$(2, 1)$	right	Move to $(2, 2)$; reward -0.1.
$(2, 1)$	wait	Stay at $(2, 1)$; reward -0.1.
$(2, 3)$	right	Reach $(2, 4)$; reward $+10$; terminate.

Check that an unfinished 40-step rollout reports truncation and stops. Do not test goal-reaching by waiting for a random policy to find the goal.
Next, test one Q-learning update using values you specify yourself:

$$Q(s, a) = 0.4, \qquad r = 1, \qquad \max_{a'} Q(s', a') = 2, \qquad \alpha = 0.2, \qquad \gamma = 0.95.$$

Compute the target and updated entry by hand, and compare with the program. This is a synthetic update test, not an environment transition.
Repeat with a truly terminal reward of 5. Then repeat the non-terminal example with the truncation flag set. Which case should ignore the next-state value?
Finally, locate these concepts in the generated program:

Concept	Function or line in your program
State and action representation	
Reward and termination	
Epsilon-greedy action choice	
TD target and Q-table update	
Training and greedy evaluation	

Think About It

A bug in the environment changes the problem the agent learns. A bug in the update changes the learning algorithm. Test them separately, even if the final plot looks reasonable.

Task 5: Train, Evaluate, and Investigate Exploration

Train the validated Q-learning agent. Plot training return and training success using a moving average, for example over 50 episodes. State the averaging window.
Evaluate from the default start with $\varepsilon = 0$ and no learning updates. Print the sequence of positions and show the trajectory on the grid. Compare its length with the route you identified in Task 1.
Compare against a uniformly random policy for 100 episodes. Use the same map, start, rewards, and 40-step cutoff.

Measure	Random policy	Greedy Q-policy
Goal reached		
Steps to goal, if successful		
Timeouts		
Invalid moves		

For the random policy, report success and timeout fractions across the 100 runs, and mean steps among successful runs if any. For the deterministic greedy policy, report its single default-start outcome, path length, and invalid moves. Use a fixed tie-breaking rule in greedy evaluation.
Answer:

(a) Why might the training curve remain noisy after a good route is learned?

(b) Does a high training return necessarily imply a good greedy policy?

(c) What is the difference between the learning rate α and the exploration probability ε?

Optional investigation. Train a fresh Q-table with no exploration, and compare it with the epsilon-greedy version. Keep the other settings fixed and repeat with another seed if time permits. Do not assume that no exploration must fail: random tie-breaking and initial values can also affect the actions tried.

6 Part II: Replace the Table with a Neural Network

Task 6: Ask an LLM to Implement and Validate DQN

Keep the single-robot environment unchanged. First write down what should change in the agent and what should remain the same.
Represent cell (r, c) by a length-25 one-hot vector with a 1 at entry $5r + c$. Use a network with:

$$25 \text{ inputs} \longrightarrow 64 \text{ ReLU units} \longrightarrow 64 \text{ ReLU units} \longrightarrow 5 \text{ linear outputs.}$$

Ask the LLM to replace the Q-table with this network.

Example prompt

Extend my validated single-robot program to use DQN in PyTorch. Keep the environment, rewards, start, goal, and discount unchanged.
Use the specified one-hot input and two 64-unit ReLU hidden layers. Output five Q-values without softmax. Use epsilon-greedy action selection, a replay buffer, and a target network.
Use a replay capacity of 20,000 transitions and random batches of 64. Initialize the target network from the online network. Collect 500 steps before learning, perform one optimizer update every four environment steps, and copy the online weights to the target every 250 optimizer updates.
Use Adam with learning rate 0.001 and mean squared TD error. Compute detached targets with `torch.no_grad()`. Use a reward-only target for termination and bootstrap at truncation.
Train for 12,000 environment steps with seed 7. Decrease epsilon linearly from 1 to 0.05 over the first 80 percent of training. Keep training and greedy evaluation separate. Explain where the recorded action is selected when computing the loss.

Before training, test the loss on a batch whose targets you can calculate:

- Non-terminal: $r = 1$ and maximum target-network next value 2.
- Terminal: $r = 5$, regardless of the next-state value.
- Truncated: $r = 1$ and maximum target-network next value 2.

Use $\gamma = 0.95$. Check that the loss compares each target with $Q_\theta(s, a)$ for the *recorded action*, and that gradients do not flow into the target.
Run DQN and compare its greedy route with the Q-table route. Locate the replay buffer, batch sampling, TD target, optimizer step, and target-network synchronization in the code.

Think About It

This small grid can be solved with a Q-table. The purpose of this step is to understand how DQN represents and learns action values. A neural network is not automatically a better choice.
The training budgets differ, so these runs do not establish which method is more sample-efficient.

7 Part III: Two Robots and Collision Avoidance

Task 7: Design and Test a Two-Robot Controller

Now place two robots in the same warehouse:

Robot	Start	Goal
A	$(2, 0)$	$(2, 4)$
B	$(2, 4)$	$(2, 0)$

They must reach their own goals without colliding. A robot reaching its own goal stays there and continues to occupy that cell.

First design the controller

Use **one controller that observes both robots and chooses both moves**. The state is the ordered pair (p_A, p_B). A joint action is (a_A, a_B).
Before prompting the LLM:

(a) Explain why observing only A's position is insufficient.

(b) Write down how you will represent both positions.

(c) Calculate the number of joint actions.

(d) Draw one possible collision-free route for both robots.

(e) Explain where one robot might need to wait or move aside.

Concatenate two length-25 one-hot vectors to obtain 50 inputs. There are $5^2 = 25$ joint actions. Encode them by $j = 5a_A + a_B$ and decode with `a_A, a_B = divmod(j, 5)`. The network now has 50 inputs and 25 linear outputs.

Specify simultaneous movement

Compute both proposed moves from the same old state. An obstacle or boundary move becomes a stay. A robot already at its goal stays, regardless of its requested action.
Then check for:

- **Same-cell collision:** the proposed positions are equal.
- **Swap collision:** the robots exchange their old positions in one simultaneous step.

Entering a waiting robot's cell is a collision. Following a robot into a cell it vacates is allowed if the final cells differ and there is no swap. On collision, retain the old positions for reporting and terminate the episode.
Use one team reward:

$$r = \begin{cases} -10, & \text{if a robot–robot collision is attempted,} \\ -0.1 - b + 5k + 5I, & \text{otherwise,} \end{cases}$$

where b is the number of invalid moves by robots not already at their goals, k is the number newly reaching their own goals, and $I = 1$ if both robots are now at their goals. A first goal arrival is rewarded only once. Collision overrides the other reward terms.
The task terminates on collision or when both robots have arrived. Truncate unfinished rollouts after 40 joint steps.

Test before training

Predict each outcome, then compare it with the generated environment:

Old A, B	A action	B action	Your prediction
$(2,1),(2,3)$	right	left	
$(2,1),(2,2)$	right	left	
$(2,1),(2,2)$	right	right	
$(2,1),(2,2)$	up	wait	

Also test a robot already at its goal. Verify that it stays there and cannot receive repeated arrival rewards.

Ask the LLM to extend your program

Give the LLM your joint-state design and all the movement and reward rules. Ask it to reuse the DQN learning code, change the input and output dimensions, and add the collision tests.
Begin with 60,000 environment steps, seed 7, and the DQN settings from Task 6. Do not ask the LLM to insert a path planner or an action filter that prevents collisions. The policy should be evaluated on the actions it actually chooses.
Print a table of the two positions after every greedy joint step. Plot both trajectories, with time labels or an animation so that simultaneous movement is clear.
Record success, collision, timeout, invalid moves, and steps to completion. Then answer:

(a) Which robot entered the passage first?

(b) Where did the other robot yield?

(c) Is avoiding collisions sufficient to count as success?

(d) How would the joint-action output size change for three robots?

Important

DQN training may stall or vary across seeds. Keep failed runs and inspect them. If time permits, try a fresh run with another seed or a larger budget. If neither run succeeds, complete the tests and explain the failure; do not replace the learned route with a planned path and report it as learning.
A robot that waits forever can avoid collisions while failing the delivery task.

8 Evaluation and Reflection

Task 8: Evaluate the Agents and the LLM's Contribution

Summarize the three controllers. Use greedy evaluation with learning disabled.

Measure	Q-table, one robot	DQN, one robot	DQN, two robots
Budget and seed			
Success / collision / timeout			
Steps, if successful			
Invalid moves			

Test at least one additional start without retraining. For one robot, try $(0, 0)$. For two robots, try A at $(0, 0)$ and B at $(4, 4)$. Keep the map and goals unchanged.
Explain whether the policy works from these starts. A policy trained from one fixed start need not work from another. Repeating a deterministic greedy rollout from the same start does not create independent evaluation trials.
Do not compare the raw returns of the one-robot and two-robot tasks as if they had the same reward definition.
Reflect on the LLM's contribution:

1. Which parts of the design did you specify before requesting code?
2. Did the LLM change any rewards, stopping rules, or algorithm details?
3. Which errors did your tests reveal?
4. Did the code use a replay buffer and target network as requested?
5. Which generated functions can you now explain without consulting the LLM?
6. What did you change, and what evidence shows the change was correct?

Think About It

Separate what the program was asked to do, what its code implements, and what your experiments demonstrate. The LLM's explanation is not a substitute for checking the program.

8.1 Optional: Connect the Exercise to RLHF

Suppose people compare pairs of recorded trajectories and prefer one because it avoids close encounters or unnecessary waiting.

Explain:

1. what feedback people would provide;
2. what a learned reward model would predict;
3. how an RL agent could use that reward model.

Does editing the hand-written collision penalty train a reward model? Explain why this is different from learning a reward from preferences. No reward model or RLHF training is required in this laboratory.

9 Submission

If you are asked for a submission, provide a short report containing:

1. your single-robot MDP and agent design;
2. the final Python programs for Q-learning and DQN;
3. the important LLM prompts and corrections;
4. the environment and TD-update test results;
5. training plots and greedy trajectories;
6. the two-robot state, action, reward, and collision specification;
7. the comparison table and additional-start results;
8. short answers to the reflection questions.

State package versions, seeds, and training budgets so that another student can reproduce your work. Report unsuccessful experiments honestly.

The Central Lesson

The scientific ideas are learning from reward and estimating action values. Q-learning and DQN provide algorithms for these ideas. The LLM helps translate your design into code.
You remain responsible for specifying the problem, understanding the update, testing the program, and explaining the evidence.

10 Reading

The course lectures on MDPs and Q-learning, Deep RL, and RLHF provide the background for this exercise.

For further reading:

- *Q-learning.* Machine Learning 8, 279–292 (1992).
  https://doi.org/10.1007/BF00992698
- *Human-level control through deep reinforcement learning.* Nature 518, 529–533 (2015).
  https://doi.org/10.1038/nature14236
- *Deep Reinforcement Learning from Human Preferences.* Advances in Neural Information Processing Systems 30 (2017).